

TP 7 : Corrections SAST

RBAC/ABAC & Pipeline CI/CD - Partie 2

Travail Pratique N°7



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction	3
2	Analyse détaillée et corrections appliquées	3
2.1	Injection NoSQL	3
2.2	Secret JWT faible	3
2.3	Credentials hardcodés	3
2.4	Rate limiting	4
3	Résultats npm audit	4
4	Intégration dans le pipeline CI/CD	4
5	Contrôle d'accès : RBAC + ABAC	4
5.1	RBAC (Role-Based Access Control)	4
5.2	ABAC (Attribute-Based Access Control)	5
6	Conclusion	5
	Captures d'écran	6

1 Introduction

Contexte - Partie 2

Cette seconde partie détaille les corrections appliquées suite à l'analyse SAST, l'intégration dans le pipeline CI/CD, et les mécanismes de contrôle d'accès (**RBAC** et **ABAC**) mis en place.

2 Analyse détaillée et corrections appliquées

2.1 Injection NoSQL

Risque : Sans validation des entrées utilisateur, un attaquant peut injecter des opérateurs MongoDB malveillants (ex : `{ $gt: '' }`) dans les requêtes, contournant ainsi l'authentification ou exfiltrant des données.

Code vulnérable détecté :

```
1 router.get('/search', authenticate, async (req, res) => {
2   const docs = await Document.find({ title: req.query.title });
3   // Entree non validee
4 });
```

Correction appliquée :

```
1 router.get('/search', authenticate, async (req, res) => {
2   const title = mongoSanitize.sanitize(req.query.title);
3   if (typeof title !== 'string' || title.length > 200) {
4     return res.status(400).json({ error: 'Titre invalide' });
5   }
6   const docs = await Document.find({ title: { $regex: title,
    $options: 'i' } });
```

La bibliothèque `express-mongo-sanitize` a été intégrée globalement dans `app.js` pour sanitizer automatiquement toutes les requêtes entrantes.

2.2 Secret JWT faible

Risque : Un secret JWT court ou prévisible peut être cracké par force brute, permettant à un attaquant de forger des tokens valides.

Correction appliquée : Utilisation d'un secret généré de manière sécurisée (256+ bits) stocké dans les variables d'environnement.

2.3 Credentials hardcodés

Risque : Des identifiants présents dans le code source sont exposés à quiconque accède au dépôt.

Correction appliquée : Tous les identifiants sont externalisés dans des variables d'environnement via `.env` (exclu du dépôt par `.gitignore`) et les variables CI/CD GitLab.

2.4 Rate limiting

Risque : Sans limitation du nombre de requêtes, l'API est vulnérable aux attaques par force brute et DDoS.

Correction appliquée : La bibliothèque `express-rate-limit` a été intégrée avec une limite de 100 requêtes par fenêtre de 15 minutes pour toutes les routes `/api/`.

3 Résultats npm audit

L'audit des dépendances npm n'a révélé aucune vulnérabilité de niveau **high** ou **critical** dans les 398 packages analysés. Les warnings concernent des packages dépréciés qui ne représentent pas de risques de sécurité immédiats.

Niveau	Nombre	Action requise
Critical	0	Aucune
High	0	Aucune
Medium	0	Aucune
Low	0	Aucune
Info	Warnings dépréciations	Non critique

4 Intégration dans le pipeline CI/CD

L'analyse SAST a été entièrement automatisée dans le pipeline GitLab CI/CD. Le stage **sast** comprend trois jobs exécutés en parallèle :

Job	Image	Commandes
ast-semgrep	returntocorp/semgrep :latest	semgrep scan --config p/nodejs --config p/jwt
ast-njsscan	python :3.11-slim	pip install njsscan && njsscan src/
dependency-check	node :18-alpine	npm audit --audit-level=high

Tous les jobs SAST sont configurés avec `allow_failure: true` pour ne pas bloquer le pipeline en cas de détection de vulnérabilités mineures, tout en générant des artifacts (rapports JSON) téléchargeables pour analyse.

5 Contrôle d'accès : RBAC + ABAC

L'application implémente deux couches de contrôle d'accès :

5.1 RBAC (Role-Based Access Control)

- **Admin** : Accès total à tous les documents et utilisateurs
- **Editor** : Peut créer, modifier et supprimer ses propres documents
- **Viewer** : Accès en lecture seule aux documents

5.2 ABAC (Attribute-Based Access Control)

- Accès basé sur les attributs des documents (département, niveau de confidentialité)
- Politiques fines basées sur le contexte (heure, localisation)

6 Conclusion

Bilan de la sécurité

L'analyse SAST de l'application **secure-doc-manager** démontre une bonne posture de sécurité. Les principales mesures de sécurité implémentées sont :

- Protection contre les injections NoSQL via **express-mongo-sanitize**
- Authentification JWT avec secret de 256+ bits stocké en variable d'environnement
- Hachage des mots de passe avec **bcryptjs** (12 rounds)
- Rate limiting sur toutes les routes API
- Contrôle d'accès en couches : RBAC + ABAC
- Principe du moindre privilège appliqué aux utilisateurs MongoDB
- Pipeline CI/CD avec analyse SAST automatique à chaque commit
- Aucune dépendance vulnérable critique détectée

Fin du rapport TP7

Captures d'écran

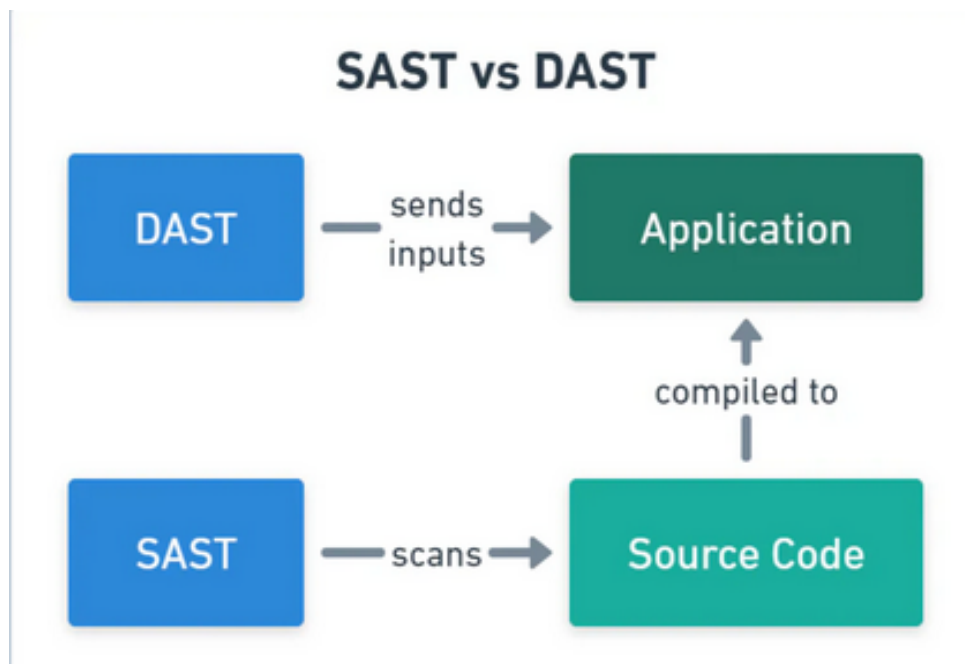


FIGURE 1 – Figure 1